Gabarito (professor): Lista de Exercícios de Tabelas Hash

Oct 28, 2026 · @Alexandre Gomes

Soluções das seis questões da lista dos alunos, com o custo de cada uma e os erros mais comuns na correção. Todos os códigos foram executados no Node e as saídas abaixo são as reais. Não distribua este documento antes da tentativa.

#	Problema	Estrutura	Sem hash	Com hash
1	Detectar duplicados	`Set`	O(n²)	O(n)
2	Frequência de palavras	`Map`	O(n·k)	O(n)
3	Two Sum	`Map`	O(n²)	O(n)
4	Primeiro caractere único	`Map`	O(n²)	O(n)
5	Agrupar anagramas	`Map`	O(n²) comparações	O(n·k log k)
6	Fibonacci com memoização	`Map`	O(2^n)	O(n)

Em "k" lê-se: o número de palavras distintas (exercício 2) ou o tamanho de cada palavra (exercício 5).

Exercício 1: Detectar duplicados

```
function temDuplicado(arr) {
  const vistos = new Set();
  for (const x of arr) {
    if (vistos.has(x)) return true;
    vistos.add(x);
  }
  return false;
}
console.log(temDuplicado([1, 2, 3, 1])); // true
console.log(temDuplicado([1, 2, 3]));    // false
```

Correção: se o aluno usou dois laços aninhados, está correto mas O(n²). Pergunte qual estrutura permite responder "já vi este valor?" em O(1). Alternativa válida e curta: `new Set(arr).size !== arr.length`.

Exercício 2: Frequência de palavras

```
function frequenciaPalavras(texto) {
  const freq = new Map();
  const palavras = texto.toLowerCase().match(/\p{L}+/gu) || [];
  for (const p of palavras) freq.set(p, (freq.get(p) || 0) + 1);
  return freq;
}
console.log(frequenciaPalavras("O gato viu o rato e o rato viu o gato"));
// Map(5) { 'o' => 4, 'gato' => 2, 'viu' => 2, 'rato' => 2, 'e' => 1 }
```

Correção: erro comum é esquecer o `toLowerCase()`, que separa "O" e "o". Outro é usar `split(" ")`, que leva a pontuação junto com a palavra. A expressão `\p{L}+` com a flag `u` pega letras com acento.

Exercício 3: Two Sum

```
function twoSum(nums, alvo) {
  const vistos = new Map(); // valor -> índice
  for (let i = 0; i < nums.length; i++) {
    const complemento = alvo - nums[i];
    if (vistos.has(complemento)) return [vistos.get(complemento), i];
    vistos.set(nums[i], i);
  }
  return null;
}
console.log(twoSum([2, 7, 11, 15], 9)); // [0, 1]
console.log(twoSum([3, 2, 4], 6));      // [1, 2]
console.log(twoSum([1, 2], 10));        // null
```

Correção: a solução ingênua tem dois laços aninhados em $O(n^2)$. Aqui o `Map` responde "já vi o complemento?" em $O(1)$. Erro comum: guardar o número antes de consultar o complemento, o que faz um número se combinar com ele mesmo (por exemplo, `twoSum([3], 6)` não deve achar nada).

Exercício 4: Primeiro caractere não repetido

```
function primeiroUnico(s) {
  const cont = new Map();
  for (const c of s) cont.set(c, (cont.get(c) || 0) + 1);
  for (const c of s) if (cont.get(c) === 1) return c;
  return null;
}
console.log(primeiroUnico("abacabad")); // c
console.log(primeiroUnico("aabb"));     // null
```

Correção: duas passadas: a primeira conta, a segunda procura na ordem original. Se o aluno iterar sobre o `Map` na segunda passada, também funciona, porque o `Map` mantém a ordem de inserção.

Exercício 5: Agrupar anagramas

```
function agruparAnagramas(palavras) {
  const grupos = new Map();
  for (const p of palavras) {
    const chave = p.split("").sort().join(""); // "amor" e "roma" viram
"amor"
    if (!grupos.has(chave)) grupos.set(chave, []);
    grupos.get(chave).push(p);
  }
  return [...grupos.values()];
}
console.log(agruparAnagramas(["amor", "roma", "ramo", "gato", "toga",
"casa"]));
// [ [ 'amor', 'roma', 'ramo' ], [ 'gato', 'toga' ], [ 'casa' ] ]
```

Correção: bom fechamento da aula: os anagramas que quebraram a primeira função de hash (a soma dos códigos) agora viram a chave da solução. Quem comparar todos os pares de palavras fez $O(n^2)$.

Exercício 6: Fibonacci com memoização

```
function fibMemo(n, memo = new Map()) {
  if (n <= 1) return n;
  if (memo.has(n)) return memo.get(n);
  const r = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
  memo.set(n, r);
  return r;
}
console.log(fibMemo(10)); // 55
console.log(fibMemo(50)); // 12586269025
```

Correção: sem a memoização, `fib(50)` recursivo recalcularia os mesmos valores bilhões de vezes e levaria minutos. Para o desafio, rode `fib(40)` sem `Map` com `console.time` para a turma sentir a espera.

Desafio para casa: keys() e maiorBalde()

O que esperar da entrega:

```
keys() {
  const chaves = [];
  for (const balde of this.tabela) {
    for (const [k] of balde) chaves.push(k);
  }
  return chaves;
}

maiorBalde() {
  let maior = 0;
  for (const balde of this.tabela) {
    if (balde.length > maior) maior = balde.length;
  }
  return maior;
}
```

- Resultado conferido com `"aluno0"` até `"aluno999"` inseridas em uma tabela de tamanho inicial 11: com o `_rehash` ativo, o tamanho final é 1535 e o maior balde tem 4 itens; sem o rehash, a tabela fica com tamanho 11 e o maior balde tem 92 itens. `keys()` devolve as 1000 chaves nos dois casos.
- Conclusão esperada: sem rehash, as listas crescem e a busca vira quase uma busca linear dentro do balde (O(n) com n/11 itens por lista). Com rehash, as listas ficam curtas e o custo médio volta a ser O(1).

- Erro comum: percorrer `this.tabela` esquecendo que cada posição é uma lista de pares, e devolver os pares em vez das chaves. A chave é o primeiro elemento do par (`par[0]`).